RIT | جامعة روتشستر الأمريكية للتكنولوجيا في دبي
A Global American University in Dubai

Department of Electrical Engineering and Computing Sciences	
Academic Year	2026-2027
Semester	○ Fall ◉ Spring ○ Summer
Course Code - Name	**GCIS124 – Software Development and Problem Solving II**
Instructor	
Exam	○ Quiz ◉ Assignment ○ Mid ○ Final ○ Other Assignment 2
Duration	9 days
Date	Friday, Feb 27, 2026 till Sunday, 8 March, 2026

- **The assessment duration is 9 days.**
- **The assessment is a group activity and published as a homework; which has to be submitted on MyCourses. Every group will be assessed by Viva and class presentation.**
- **You will use AI tools for Part B of the Assignment. For Part A, no AI tools are allowed.**
- **Any code used from unapproved sources and the code concepts that we did not learn during classes yet will be marked as 0.**
- **Any member/group unable to present on designated date(s) will be marked 0. No extended date will be assigned to the group/member for viva and/or presentation.**

Student ID: _____

Student Name:

CLOs:

1. Utilize the basic constructs of a selected contemporary high-level programming language
2. Apply problem-solving skills to develop solutions for computing problems of varying complexity
3. Implement algorithms and data structures to solve computing problems of different difficulty levels.
4. Demonstrate fundamental software engineering concepts, including understanding needs, software design, solution testing, and incremental development.

◉ **Check this box if this assessment maps to ONE and only ONE CLO**

CLO	Max Grade	Achieved Grade
CLO 2	**10**	

Assignment 2: GreenPrint GUI & AI Comparison

Software Dev. & Problem Solving II — GCIS-124

GreenPrint — Part II

GUI-Based Carbon Tracker with JavaFX + AI-Assisted Rebuild & Comparison

Overview

This assignment has two parts. In Part A, you will convert your Assignment 1 CLI application into a fully functional Graphical User Interface (GUI) using JavaFX. The GUI must support real-time emission visualization, input validation, simulated offset transaction processing, and persistent data logging. In Part B, you will rebuild the GUI (only the GUI — not the model classes) using Generative AI tools, then write a structured comparison between your manually-built GUI and the AI-generated GUI. This teaches you both how to build professional GUIs and how to critically evaluate AI-generated code.

Objectives

Upon successful completion, students will be able to:

- Translate a CLI application into an interactive, event-driven GUI system using JavaFX
- Design multi-tabbed interfaces with real-time data visualization and input validation
- Implement file-based data persistence with exception handling
- Save and restore application state between sessions
- Use Generative AI tools to rebuild a GUI and critically compare the results
- Document AI prompts, evaluate AI output quality, and identify where human intervention was needed

Technology Requirements

- Programming Language: Java 17 or higher
- GUI Framework: JavaFX 17 or higher
- Layout Managers: JavaFX layout classes (BorderPane, VBox, HBox, GridPane, TabPane, etc.)
- No use of Swing or AWT components is permitted
- AI tools for Part B: ChatGPT, GitHub Copilot, Gemini, Claude, Codeium, or similar (document which tool you use)

Part A: Manual GUI Implementation

Build the entire GUI yourself without AI assistance. The GUI must use a TabPane as its main container, with three tabs described below. Your model classes (EmissionSource, subclasses, FootprintTracker) from Assignment 1 remain unchanged — the GUI is the view/controller layer on top of them.

1. Interface & System Architecture

The system must be modularized with clear responsibility separation. You must have:

- **A main GUI class** that extends javafx.application.Application and serves as the entry point. It creates the TabPane and assembles the three tabs.
- **An input validator module** (a separate class or utility method) that validates user-entered emission source IDs against a pattern using regular expressions. For example, IDs must match the pattern: one uppercase letter, a hyphen, then exactly three digits (e.g. "T-001", "E-042", "F-123"). The validator returns true/false and the GUI must show a green or red indicator next to the input field in real-time as the user types.
- **A transaction handler module** (a separate class) that simulates carbon offset purchases. When the user "buys" an offset, this module calculates the cost, generates a receipt, and returns the result to the GUI for display.
- **A logger utility** (a separate class) that appends every operation (entry added, offset purchased, state saved) to a log file called greenprint_log.txt with timestamps.

a. Live Emission Dashboard Tab

This tab displays a visual overview of all emission entries currently in the system. Requirements:

- Display each emission entry as a color-coded label or rectangle inside a FlowPane or GridPane. Use green for low emissions (< 1.0 kg CO_2), yellow for medium (1.0–3.0 kg CO_2), and red for high (> 3.0 kg CO_2).
- Each colored element must display the source ID as text on it.
- When the user hovers over or clicks on an element, display the full details of that emission entry in a detail panel below the grid (or in a popup). Details include: source ID, category, user name, date, specific attributes (e.g. distance and mode for transportation), and the calculated emission value.
- The dashboard must update in real-time whenever a new entry is added from the Input tab (see below). This means after adding an entry on Tab 2, switching back to Tab 1 should show the new element immediately.
- At the top of this tab, display a summary bar showing: total entries count, total CO_2 emitted (formatted to 2 decimal places), and the name of the user with the highest footprint.

b. Input & Operations Tab

This tab allows the user to add new emission entries and search/filter existing ones. Requirements:

- A TextField for the Source ID with real-time validation. As the user types, a small Label next to it should show "✓ Valid" in green or "✗ Invalid" in red, depending on whether the current text matches the regex pattern [A-Z]-\d{3}. Use a ChangeListener on the TextField's textProperty() or a KeyEvent handler.

- A ComboBox (dropdown) for selecting the emission type: "Transportation", "Energy", or "Food".
- When the user selects a type, additional input fields must appear below (and any previous type-specific fields must disappear). For Transportation: TextFields for distance (km) and a ComboBox for mode (Car, Bus, Train, Bicycle). For Energy: TextField for kWh and ComboBox for source (Grid, Solar, Wind). For Food: ComboBox for meal type (Vegan, Vegetarian, Poultry, Beef) and TextField for number of meals.
- A TextField for the user name (e.g. "Alice") and a DatePicker or TextField for the date.
- An "Add Entry" Button. When clicked: validate all inputs, create the correct EmissionSource subclass object, add it to the FootprintTracker, log the action, and show a success message. If any input is invalid (empty fields, non-numeric where numbers are expected), show an error message in a Label styled with red text. Do NOT crash.
- A "Search by User" section: a TextField and a "Search" Button. When clicked, display all entries for that user name in a ListView or VBox below, each showing the entry's toString() output.

c. Carbon Offset Transaction Tab

This tab simulates the purchase of carbon offsets to neutralize the user's footprint. Requirements:

- Display the current total emissions at the top of the tab (pulled from FootprintTracker.getTotalEmissions()).
- A TextField where the user enters how many kg of CO_2 they want to offset.
- A ComboBox for payment method: "Credit Card", "Digital Wallet", "Campus Card".
- A "Purchase Offset" Button. When clicked: simulate a 2-second processing delay (use a PauseTransition or Timeline, NOT Thread.sleep()), then display a receipt in a TextArea or set of Labels showing: date/time, amount of CO_2 offset, cost (use a rate of $15 per 1000 kg CO_2, or your own rate — document it), payment method, and a confirmation message. Log the transaction to the log file.
- An "Offset History" section at the bottom: a ListView or VBox that shows all past offset transactions from the current session.

2. Data Persistence

All operations must be logged to greenprint_log.txt. Each log line must include:

- Timestamp (e.g. 2026-02-12 14:35:07)
- Operation type (e.g. "ENTRY_ADDED", "OFFSET_PURCHASED", "STATE_SAVED")
- Relevant details (source ID, user name, emission value, or offset amount)

The file must be opened in append mode (never overwrite). All file I/O must be wrapped in try-catch blocks.

3. Application State Persistence (Save & Resume)

The application must save its state when closed and restore it on next launch:

- When the user closes the window (or clicks a "Save & Exit" button), write all current EmissionSource entries to a file called greenprint_state.txt using a structured format (one entry per line, fields separated by a delimiter like "|" or ",").
- On startup, check if greenprint_state.txt exists. If it does, read each line, reconstruct the appropriate EmissionSource subclass object, and add it to the FootprintTracker. The dashboard should immediately show the restored entries.
- Use BufferedReader/BufferedWriter or FileReader/FileWriter. All reading and writing must include proper exception handling.

Part B: AI-Assisted GUI Rebuild & Comparison

Important: In Part B you are only rebuilding the GUI layer. You must still use your own model classes (EmissionSource, subclasses, FootprintTracker) from Assignment 1. The AI is used to generate the JavaFX GUI code (tabs, layouts, controls, event handlers, styling) — not the domain logic.

1. AI-Based GUI Reimplementation

Using a Generative AI tool of your choice (ChatGPT, Copilot, Gemini, Claude, Codeium, etc.), regenerate the entire GUI layer of your application. You must:

- Prompt the AI to create the same three-tab JavaFX interface described in Part A (Dashboard, Input, Offset Transaction).
- Save every prompt you send and every response you receive. Keep these in a file called ai_prompts_log.txt or ai_prompts_log.md.
- You are allowed (and expected) to refine your prompts. If the AI's first attempt is wrong, send a follow-up prompt correcting the issue. Document these refinement steps.
- You may modify or fix the AI-generated code to make it compile and run. Document every change you make and why it was necessary.
- The AI-generated GUI must connect to YOUR model classes — do not let the AI rewrite EmissionSource, FootprintTracker, etc.

2. Comparison Report

Write a 2–3 page report comparing your manually-built GUI (Part A) with the AI-generated GUI (Part B). The report must include the following comparison table filled in with your honest evaluation:

Criterion	Manual GUI (Part A)	AI GUI (Part B)	Evaluation Details
Code Structure & Readability			Use of classes, modularity, comments, naming conventions
GUI Layout & Visual Quality			Layout clarity, alignment, spacing, color usage, professional look
Feature Completeness			Were all three tabs fully implemented with all required features?
Input Validation & Error Handling			Real-time validation, graceful error messages, no crashes
Responsiveness & UX			Does the GUI resize well? Is the interaction flow intuitive?
Development Time			Hours spent: manual coding vs. prompting + fixing AI output
AI Prompting Quality	N/A		Were prompts specific and effective? How many iterations needed?
Required Human Edits to AI Output	N/A		Number and nature of corrections needed after AI generation

Include screenshots of both GUIs side by side for each tab. Add your own additional criteria if you wish. End the report with a paragraph reflecting on: which GUI was better overall, what AI did well, what it struggled with, and whether you would use AI for GUI development in future projects.

Grading Rubric (100%)

Part A (Manual GUI) and Part B (AI Comparison) are graded together:

Evaluation Area	Weight
Updated UML Diagram (reflecting GUI architecture)	10%
JavaFX GUI: Live Emission Dashboard Tab	20%
JavaFX GUI: Input & Operations Tab (incl. validation)	15%
JavaFX GUI: Offset Transaction Tab	10%
Data Persistence (logging + state save/resume)	10%
UI Design Quality & Usability	5%
AI-Generated GUI Completeness & Functionality	10%
AI Prompt Log (quality, refinement, documentation)	5%
Comparison Report & Table (depth, honesty, screenshots)	10%
Code Structure & Documentation	5%

Deliverables

1. Updated UML diagram (PNG or PDF) reflecting all classes including GUI architecture
2. Complete Java source files for Part A (your manual GUI), organized in packages (e.g. ui, model, util)
3. Complete Java source files for Part B (AI-generated GUI), in a separate package or folder (e.g. ui.ai)
4. ai_prompts_log.txt — full record of all AI prompts and responses
5. Comparison report (PDF or DOCX) with filled comparison table and screenshots
6. greenprint_log.txt — sample log file with at least 10 entries
7. At least 3 screenshots of each GUI (6 total: 3 manual, 3 AI)